DIAGRAMAS DO SISTEMA

Projeto: Gestão de Ordens de Serviço Empresarial

1 Diagrama de classes

O diagrama de classes representa a estrutura lógica do sistema, apresentando as principais entidades, seus atributos, responsabilidades e relacionamentos.

Como o projeto atual possui uma estrutura **multiempresa**, o modelo precisa representar não apenas usuários e Ordens de Serviço, mas também empresas, setores, categorias e demais elementos necessários para controlar o funcionamento da plataforma.

As principais classes consideradas são: Empresa, Usuário, Setor, Categoria, OrdemServico, Notificacao e Historico.

1.1 Classe Empresa

A classe Empresa representa uma organização cadastrada na plataforma.

Atributos principais:

- id;
- nome;
- código;
- administradorPrincipalId;
- status;
- dataCriacao;
- configuracoes.

Uma empresa pode possuir diversos usuários, setores, categorias, Ordens de Serviço, notificações e registros de histórico.

1.2 Classe Usuário

A classe Usuário representa as pessoas cadastradas em uma empresa.

Atributos principais:

- id;
- empresaId;
- nome;
- email;
- senhaHash;
- setorId;

- funcao;
- permissoes;
- status;
- dataCriacao.

O usuário poderá exercer diferentes funções, como Colaborador, Gestor, Administrador ou Administrador Principal.

1.3 Classe Setor

A classe Setor representa os departamentos ou áreas da empresa.

Atributos principais:

- id;
- empresaId;
- nome;
- descricao;
- status.

Uma empresa poderá possuir vários setores, e cada usuário poderá estar vinculado a um ou mais setores conforme as regras de permissão definidas pelo sistema.

1.4 Classe Categoria

A classe Categoria representa a classificação das Ordens de Serviço.

Atributos principais:

- id;
- empresaId;
- nome;
- descricao;
- status.

As categorias poderão possuir subcategorias, permitindo uma classificação mais detalhada das solicitações.

1.5 Classe OrdemServico

A classe OrdemServico é uma das principais entidades do sistema, representando a solicitação interna registrada por um usuário.

Atributos principais:

- id;
- empresaId;

- solicitanteId;
- setorSolicitanteId;
- setorResponsavelId;
- responsavelId;
- categoriaId;
- subcategoriaId;
- titulo;
- descricao;
- prioridade;
- status;
- sigilosa;
- dataCriacao;
- prazo;
- dataAtualizacao;
- dataConclusao;
- anexos.

A Ordem de Serviço deverá permanecer vinculada à empresa que a originou, permitindo que o backend controle o isolamento das informações.

1.6 Classe Notificacao

A classe Notificacao representa as mensagens e alertas enviados aos usuários.

Atributos principais:

- id;
- empresaId;
- usuarioId;
- ordemServicoId;
- tipo;
- mensagem;
- lida;
- dataCriacao.

As notificações poderão informar a criação de uma OS, encaminhamento, alteração de status, atualização, conclusão ou aproximação de prazo.

1.7 Classe Historico

A classe Historico representa as alterações realizadas nas Ordens de Serviço.

Atributos principais:

- id;
- empresaId;

- ordemServicoId;
- usuarioId;
- acao;
- descricao;
- dataHora.

O histórico permitirá identificar as principais alterações realizadas durante o ciclo de atendimento.

1.8 Relacionamentos principais

Os principais relacionamentos podem ser representados da seguinte forma:

- Uma empresa possui vários usuários;
- Uma empresa possui vários setores;
- Uma empresa possui várias categorias;
- Uma empresa possui várias Ordens de Serviço;
- Um setor pode possuir vários usuários;
- Um setor pode receber várias Ordens de Serviço;
- Um usuário pode criar várias Ordens de Serviço;
- Uma Ordem de Serviço possui um solicitante;
- Uma Ordem de Serviço possui um setor solicitante;
- Uma Ordem de Serviço possui um setor responsável;
- Uma Ordem de Serviço pode possuir um responsável pelo atendimento;
- Uma categoria pode estar associada a várias Ordens de Serviço;
- Uma Ordem de Serviço pode possuir vários registros de histórico;
- Um usuário pode receber várias notificações.

1.9 Representação simplificada



└── 1:N ── NOTIFICAÇÃO

Esse modelo representa melhor o funcionamento atual da plataforma, principalmente por considerar o vínculo de cada informação com uma empresa específica.

2 Diagrama de sequência

O diagrama de sequência representa a interação entre os usuários, a interface, a API e o armazenamento de dados durante a execução de uma funcionalidade.

Para o projeto atual, um dos principais fluxos é a **abertura de uma Ordem de Serviço**.

Inicialmente, o colaborador acessa a plataforma e realiza sua autenticação. O backend valida suas credenciais e identifica a empresa e as permissões associadas à conta.

Depois da autenticação, o colaborador acessa o formulário de nova Ordem de Serviço e informa os dados da solicitação, como título, descrição, categoria, prioridade, setor responsável e, quando necessário, indicação de sigilo.

A solicitação é enviada pela interface para a API REST desenvolvida com Node.js e Express.js. O backend valida os dados, verifica as permissões do usuário e associa a Ordem de Serviço à empresa correspondente.

Depois disso, a OS é armazenada no **db.json**, o histórico inicial é criado e as notificações correspondentes são registradas.

Por fim, o sistema informa ao colaborador que a Ordem de Serviço foi criada e apresenta o identificador da solicitação.

2.1 Fluxo principal

1. Colaborador acessa o sistema;
2. Realiza login;
3. Backend valida as credenciais;
4. Sistema identifica empresa e permissões;
5. Colaborador acessa “Nova Ordem de Serviço”;
6. Preenche os dados da solicitação;
7. Sistema envia os dados para a API;
8. Backend valida os dados e as permissões;
9. Backend cria a Ordem de Serviço;
10. Dados são armazenados no db.json;
11. Sistema registra o histórico;
12. Sistema gera as notificações necessárias;
13. Sistema retorna a confirmação ao colaborador.

2.2 Representação simplificada

Colaborador → Interface: Login
Interface → API: Enviar credenciais
API → db.json: Validar usuário
db.json → API: Usuário válido
API → Interface: Autenticação autorizada

Colaborador → Interface: Preencher Ordem de Serviço
Interface → API: Enviar dados da OS
API → API: Validar dados e permissões
API → db.json: Salvar Ordem de Serviço
db.json → API: Confirmação

API → db.json: Registrar histórico
API → db.json: Criar notificações
API → Interface: OS criada com sucesso
Interface → Colaborador: Exibir confirmação

3 Estrutura de dados e representação do armazenamento

Como o projeto atual não utiliza um banco de dados relacional como MySQL, a representação do armazenamento deve ser adaptada à estrutura utilizada na implementação.

Os dados serão armazenados em um arquivo **db.json**, organizado em coleções de objetos.

Entre as principais coleções estão:

- empresas;
- usuarios;
- setores;
- categorias;
- ordensServico;
- notificacoes;
- historico;
- auditoria.

3.1 Coleção empresas

empresas
- id
- nome
- codigo
- administradorPrincipalId
- status

- dataCriacao
- configuracoes

3.2 Coleção usuarios

usuarios
- id
- empresaId
- nome
- email
- senhaHash
- setorId
- funcao
- permissoes
- status
- dataCriacao

3.3 Coleção setores

setores
- id
- empresaId
- nome
- descricao
- status

3.4 Coleção categorias

categorias
- id
- empresaId
- nome
- descricao
- status

3.5 Coleção ordensServico

ordensServico
- id
- empresaId
- solicitanteId
- setorSolicitanteId
- setorResponsavelId

- responsavelId
- categoriaId
- subcategoriaId
- titulo
- descricao
- prioridade
- status
- sigilosa
- dataCriacao
- prazo
- dataAtualizacao
- dataConclusao
- anexos

3.6 Coleção notificacoes

notificacoes
- id
- empresaId
- usuarioId
- ordemServicoId
- tipo
- mensagem
- lida
- dataCriacao

3.7 Coleção historico

historico
- id
- empresaId
- ordemServicoId
- usuarioId
- acao
- descricao
- dataHora

3.8 Coleção auditoria

auditoria
- id
- empresaId
- usuarioId

- acao
- entidade
- entidadeId
- descricao
- dataHora

4 Considerações sobre a modelagem

A modelagem atual precisa refletir o funcionamento real da plataforma. Por esse motivo, a estrutura foi ampliada em relação ao modelo inicial do sistema Resolva Já.

A principal alteração é a introdução da entidade **Empresa**, que funciona como elemento central de isolamento dos dados. Todas as principais informações relacionadas ao funcionamento da organização deverão estar vinculadas à empresa correspondente.

Além disso, a modelagem passa a considerar diferentes perfis de usuários, setores, responsáveis pelo atendimento, notificações, histórico, auditoria, prioridades, prazos e solicitações sigilosas.

Mesmo utilizando um arquivo JSON para armazenamento, os conceitos de relacionamento entre entidades continuam sendo importantes para organizar os dados e representar corretamente as regras do sistema.

Dessa forma, a modelagem apresentada serve como base para a implementação das funcionalidades e para a definição dos diagramas UML do projeto.